本章路線

這章要回答的核心問題是：檔案系統介面如何以可組合的物件、名稱與權限，表達持久資料的存取規則？

前置知識。 block 儲存、process 與 file descriptor、基本權限模型。

- 區分 file metadata、資料與 handle 狀態
- 比較 sequential、direct、indexed access
- 分析 mount、share 與 protection 的邊界

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

檔案、名稱、descriptor：三種不同身分

採 Linux/POSIX 一般本機 regular file 的一般讀寫 open，排除 O_PATH、裝置特殊行為及額外 LSM 撤權。物件身分用 (filesystem,inode)，例如 (F1,42)，不是字串 `/data/a`。同一物件可有兩個 hard-link 名稱；同一路徑也可先後指向不同 inode。此識別只涵蓋物件存活期間；回收後 inode 號碼可被重用，不能以這對數值跨任意時間保證同一物件。

層	保存的狀態	可共享者
directory entry	name → inode；名稱在此	hard link 增加另一個 entry
inode／file object	size、owner、mode、資料定位、link count	所有指向同一物件的 opens
open file description，OFD	current offset、access mode、status flags	dup／fork 繼承的 descriptors
process descriptor table	小整數 fd → OFD；FD_CLOEXEC	各 process 有自己的 table（本例 fork）

權限在 open 建立已授權 OFD；後續 read/write 仍檢查 descriptor 的 access mode。chmod 改 DAC 不會自動讓本機已開 fd 失效。其他保護機制另有規則。

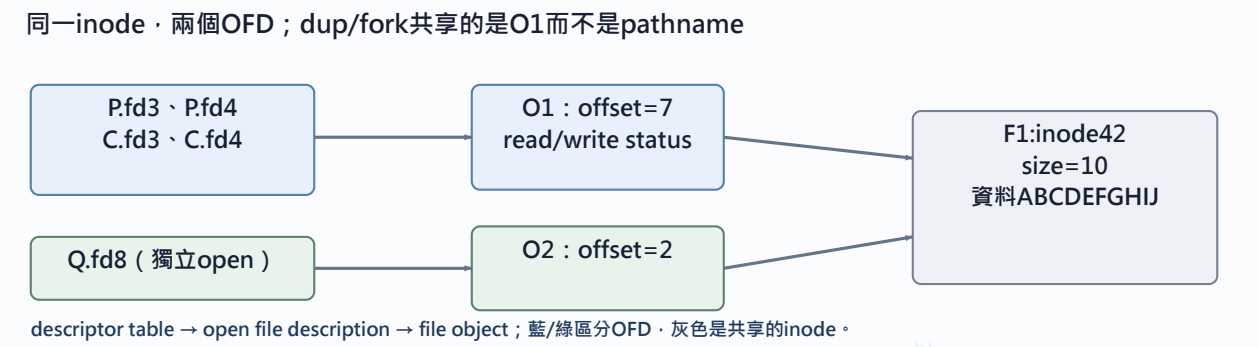
共享 offset 的每一步：資料為 ABCDEFGHIJ

P 先 `fd3=open(F)` 得 O1，再 `fd4=dup(fd3)`；Q 獨立 `fd8=open(F)` 得 O2。接著 P fork C，C.fd3/4 同指 O1。下列為指定的序列執行，避免把歷史 Linux 並行 offset 更新視為本例保證。

步驟	回傳資料	O1.offset	O2.offset
初始化	—	0	0
P read(fd3,3)	ABC	3	0
P read(fd4,2)	DE	5	0
Q read(fd8,2)	AB	5	2
C read(fd3,2)	FG	7	2
P pread(fd3,2,offset=1)	BC	7	2

每次成功 read 長度 k，使對應 OFD 的 offset 加 k；pread 使用自己的 offset、不改 O1。兩次獨立 open 的 offset 不會因 inode 相同而合併。

共享 offset 的每一步：資料為 ABCDEFGHIJ （續）



unlink、truncate、rename 的存活對象

普通檔 F1:42 初始有 `/a`、`/b`，link count=2；O1 已 open。unlink(a) 後 count=1；unlink(b) 後 count=0，但 O1 仍可讀。最後 close 且無其他持有者，才能回收。一般模型須同時區分名稱引用與開啟引用；實際還可能有 mmap 等引用。

truncate(F,0) 保留物件與名稱，size 變0；既有 OFD 的 offset 不必歸零。若 offset=7 再讀已空檔，回傳0，不是把 offset 改成0。

在同一掛載點、權限與類型允許、tmp 和 target 是不同物件且 rename(tmp,target) 成功時，呼叫原子替換名稱：新 lookup(target) 指向新 inode，已 open 舊 target 的 fd 仍指舊 inode。若兩名稱已是同一 inode 的 hard links，Linux rename 成功但不改變任何名稱；跨不同 mount 即使底層是同一 filesystem 也可能以 EXDEV 失敗。原子名稱變更只解決執行期間的可見性；斷電後能否保留，另需持久化協定。

副檔名是應用慣例，不能證明內容格式或允許執行。Linux 執行還檢查權限、格式／interpreter 等；名稱改成 `.exe` 不會把任意 bytes 變成合法程式。

sequential、direct、indexed：邏輯介面而非實體位置

方法	指定什麼	需要維持什麼
sequential	下一段	current position；rewind／seek 規則
direct	相對 record／block number	固定長度或可換算的 record 邊界
indexed	key	key→record／block 的索引與更新一致性

固定長度 L、record 編號從0開始，record j 起始 byte=jL。相對 block number 也從檔案開頭算；不能直接當磁碟 LBA，實體映射由第12章處理。

例 L=32B、block=1024B，每 block32筆；record77 的 byte offset=2464，file block2、block內 416。1000筆共32000B，需要ceil(32000/1024)=32 blocks，末 block有8筆、256B，內部碎片 768B。此例不含 sparse/compression/inline data。

sparse index 的範圍判斷與 I/O 計數

1000筆依唯一遞增整數 key 0…999 排序，仍用每 block32筆。索引32 entries，entry=(first key,block pointer)，各16B，共512B。查 key77，找最大 first key≤77，得到64→block2；讀 block2 後在64…95中找77。

一般若第 i block 的鍵值區間為 [k_i,k_(i+1))，鍵排序與索引正確就保證只需查該候選 block；落在 gap 仍須比對實際 key，不能把 predecessor 命中視為資料存在。

索引已在 RAM 時，binary search 比較次數至多 floor(log2 32)+1=6，另1次 data-block I/O；索引不在 RAM 且整個索引位於1 block，則最多1次 index +1次 data I/O。這不是每個現實查詢固定2次，更不是 O(1) CPU 搜尋。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

短讀寫：不變量是已完成前綴，不是呼叫次數

要求寫6000 bytes；依指定回傳值2500、1500、2000：

呼叫	buffer起點	本次請求	成功回傳	累計done／下一offset（初始100）
1	0	6000	2500	2500／2600
2	2500	3500	1500	4000／4100
3	4000	2000	2000	6000／6100

迴圈不變量：0≤done≤N，buffer[0:done]已依序交付；下一次只送 [done:N]。成功 k>0 後 done+=k，保留前綴且縮短剩餘量；若永遠有正進展，最多 N 次1-byte成功就完成。錯誤並不保證終止成功；EINTR（未傳資料）可重試，EAGAIN 應等就緒，回傳0不可無限空轉。

read 到 EOF 可得到短資料，下一次 request>0 回傳0；count=0 的零回傳不證明 EOF。read error 後 offset 未必可推知。本表固定成功結果，沒有替錯誤亂加 offset。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

append 的原子單位與 positioned I/O

自行 lseek(SEEK_END) 再 write 不是單一原子步：P/Q 同時見 size100，皆寫 offset100，後者可覆蓋前者。O_APPEND 將「取 EOF＋該次 write」合為受介面規範的步驟；依檔案系統／協定的實作範圍判斷，尤其不能假定 NFS 模擬追加等同本機。

即使每次 append write 原子，一筆 record 分成 A1、A2 兩次呼叫，仍可能交錯 A1,B1,A2。需要整筆 record 的 framing、鎖或單 writer 才能建立更大的承諾；regular file 不是 pipe，不能套 PIPE_BUF 定理。

pread/pwrite 把 offset 作參數，可避免 lseek＋read 被別的 thread 改共享 offset；它們不自動讓重疊資料寫入互斥。本節的 pwrite 情境排除 O_APPEND；Linux 的 O_APPEND＋pwrite 有追加行為的特例。

directory 結構的能力與代價

目錄是 name→object 的映射；同一目錄名稱唯一，跨目錄可重複。搜尋／create／unlink／rename／列舉是介面，不要求底層必用線性表。

結構	可表達	需要面對
single-level	所有人同一名稱域	名稱衝突、無分組
two-level	各 user 有自己的目錄	跨 user 分享不自然
tree	路徑與子樹分組	單一父系下無直接共享子物件
DAG	多個名稱／路徑到同物件	引用計數、重複走訪
general graph	允許 cycle	終止、回收與環偵測

成功的 hard link 在同一 mounted filesystem 中新增同 inode entry，資料只有一份；Linux 跨不同 mount 可能以 EXDEV 失敗，即使底層是同一 filesystem，權限與物件類型仍須允許；symlink 本身是存路徑的另一物件，可能跨 filesystem、dangling 或循環。一般使用者不能任意 hard-link 目錄，避免破壞父子結構；概念 DAG 不是允許任意 Linux 目錄 hard link。

reference count 為何無法回收 cycle

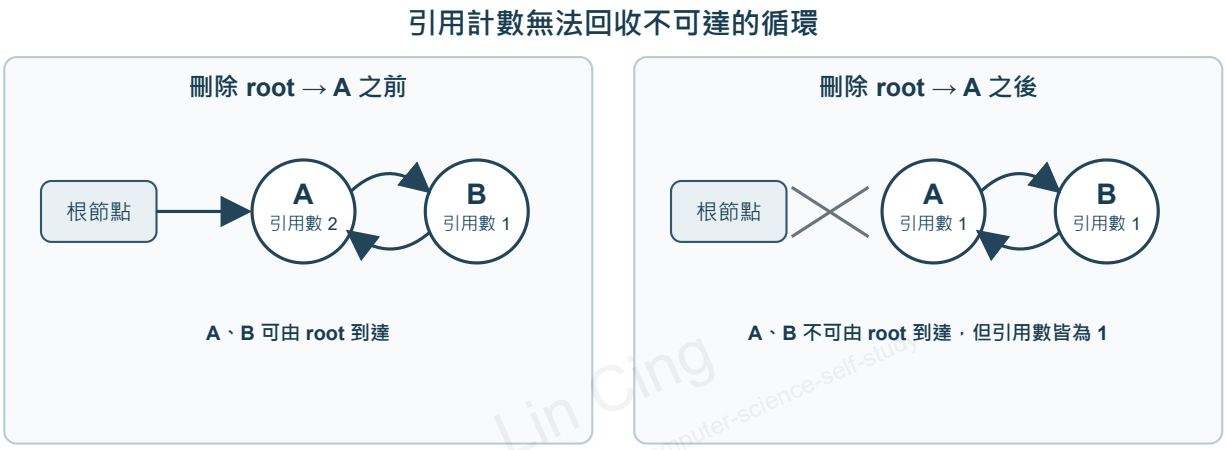
只考慮名稱／邊的抽象物件圖，不含持有中的 opens。A→B、B→A 且 root→A，計數 A=2、B=1。刪 root→A 後 A=1、B=1，兩者都不可達卻不會變0。

對有限 DAG：不可達子圖仍無 cycle，必有入度0頂點。它沒有可達頂點的入邊（否則可達），所以實際引用數為0；刪它再歸納，所有不可達頂點可被回收。cycle 沒有這個入度0起點。

traversal 用 visited set 可避免重複拜訪已遇過的物件，有限 V/E 的 DFS 為 O(V+E)；它不等於回收算法。mark/sweep 則從 roots 標記可達、回收未標記，必須固定／協調圖在觀察期間的修改。symlink 的循環限制只保證單次解析終止，不證明資料都可達。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

reference count 為何無法回收 cycle （續）



只計圖中的邊；不含 open、mmap 等其他引用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

路徑不能先作字串消去

固定 `/a/link` 的 symlink 內容為絕對 `/x/y`，所有目錄存在、可搜尋；輸入 `/a/link/../b`。

步	目前物件	待處理成分	動作
0	root	a,link,..,b	由 process root開始
1	/a	link,..,b	查 link 得 symlink
2	root	x,y,..,b	絕對target重設root，保留原suffix
3	/x/y	..,b	解析x、y
4	/x	b	從已解析物件處理..
5	/x/b	空	完成

先刪 `link/..` 會誤得 `/a/b`。相對 symlink 的 target 以「含 link 的目錄」為基準，並不是 process cwd。每次查下一成分要對目前目錄有 search(x) 權限；final symlink 是否跟隨則依操作。

路徑不能先作字串消去 （續）



TOCTOU：檢查的名字不等於操作的物件

P 先檢查 `/tmp/out` 是安全 ordinary file；攻擊者在檢查後將它改為指向敏感檔的 symlink；P 再 open 同字串，可能解析到另一物件。問題是兩次解析中間名稱圖可改，不是所有 symlink 都非法。

以已取得的可信目錄 fd 搭配 openat 可固定相對起點；O_NOFOLLOW 只處理 final symlink，不能因此聲稱中間各成分都可信。Linux 新版 openat2 的解析限制可約束整條路徑，屬後續介面，不回填為 Linux3.2已有。

操作應沿同一受保護 reference／受限解析完成。取得 fd 後的 fstat 檢查的是已開物件；仍需規定可接受的 owner/type 與 open 本身允許的副作用，不能先 O_TRUNC 再事後判斷是否安全。

mount 改的是名稱解析邊，不是把資料複製過去

F0 的 `/mnt` 原本有 `old` 且沒有 `new`；F7 root 有 `new` 且沒有 `old`。所有路徑祖先與涉及的目錄均可搜尋，且沒有其他阻擋解析的權限限制。掛 F7 到 /mnt 後，新 lookup `/mnt/new`→(F7,new)，`/mnt/old`→ENOENT；F0的old仍在，未被刪除。卸載後原名稱重新可見。

觀察	mount前	mount後
新解析 /mnt/old	F0:old	F7無old，ENOENT
新解析 /mnt/new	ENOENT	F7:new
事先open(F0:old)的fd	F0:old	仍F0:old

namespace 選擇哪張 mount map；相同字串在不同 namespace 可到不同物件。這不是 namespace 複製資料。圖中 blue=F0，green=F7，灰=被新 lookup遮住的舊內容。



mount crossing、.. 與 busy reference

在本章 Linux式模型，解析到 mountpoint 時換成掛上去的 root；在掛載 root 處解析 `..` 回到該掛載點的父目錄，而非任意停在底層filesystem root。process root 本身的 `..` 仍留在 root。

例 `/a/link` 的 target=`../b`，F7掛在 `/b`；輸入 `/a/link`：root→a→讀link→a/..=root→b→cross到F7root，至此結束。沒有輸入 filename，就不能再憑空加 `/file`。

正常 unmount 可因 cwd/open file/子掛載等活躍引用回報 busy，依系統規則判斷。Linux lazy detach 是先從 namespace 解連，物件最後引用消失再清理；不等於立刻讓所有 fd無效、也不保證 pending I/O成功。

將 mountpoint 當成 symlink 是錯誤模型：mount由namespace表決定，symlink是目錄中的一個有target字串的物件；兩者能在同一次pathwalk先後發生，但不能混為同一entry種類。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

共享要分開指定 identity、ordering、visibility 與 durability

同一 user name 在兩台主機不必有相同 UID；AUTH_SYS式數值UID映射若不一致，可能對錯人授權。網路位址／UID宣告本身也不能代替可信驗證。書中的 NIS／LDAP／Active Directory 屬命名與身分基礎設施案例；LDAP是目錄協定，不能只因使用LDAP就推得連線加密或正確驗證。

檔案協定需分開回答：誰有權操作、兩個操作如何排序、對方何時重讀到新值、何時扛得住 crash。NFSv3基本檔案協定不靠OPEN狀態定位每次READ/WRITE，但仍有cache與穩定寫入語意；NFSv4加入stateful機制。不能把「stateless」推成沒有任何狀態或自動安全。

同路徑可在不同mount看到不同file；同file也可因cache／session規則暫時看到不同內容。成功 write、成功lock、其他client看見、媒體持久化是四個不同事件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一歷史，比較 UNIX可見性與session semantics

固定初值0、A/B都先open。A寫1；B在A close前讀；A close；B在自己既有session再讀；C此時才open讀。每步依序，忽略錯誤／data race；「UNIX欄」是本章單一一致file image的抽象，不宣稱所有網路cache已符合。

同一段歷史：即時可見與 session snapshot

事件	單一共享 image	Session snapshot
A、B open	A、B 見 0	A、B 各存 0
A write 1	image 變 1	A 私有 1，B 仍 0
B read	讀到 1	讀到 0
A close / 發布	image 仍為 1	公開版本變 1
B read（不重開）	讀到 1	舊 snapshot 仍讀 0
C open / read	讀到 1	新 snapshot 讀 1

範例固定依序執行；session 的多寫者衝突另定，不代表所有網路檔案系統。

session模型讓已開instance不被後來close改動；多writer的衝突／last-close規則須另定。immutable shared file則禁止內容與其唯一名稱被修改／重用，降低cache一致性問題，代價是更新以新物件／名稱表達。

advisory lock 的保證是有條件的

A對byte區間[0,100)持exclusive lock；B要同區間shared/exclusive須等，C對不重疊[100,200)可進。半開區間重疊當 max(start)<min(end)；[0,100)與[100,200)不重疊。

t	A	遵守協定的B	未拿lock的C
0	取得[0,100) X	—	—
1	寫version8	申請同區間，等待	可直接寫同資料（advisory不阻止）
2	fsync成功	仍等	可能破壞A/B預期
3	unlock	取得lock，依cache協定重驗／讀	—

互斥證明依賴「所有衝突操作先取得相容鎖」：若兩writer同時在區間內，將同時持互斥X鎖，與鎖服務規則矛盾。C不拿鎖時前提失效，證明不能套用。單獨鎖不保證cache invalidation或durability；必須先指定共享檔案系統契約。

發布新版本：atomic rename 不等於 durable rename

固定Linux一般本機filesystem，支援同掛載點的 rename 原子替換及 directory fsync，裝置遵守 flush；tmp 與 target 在同目錄且為不同物件，並假設權限、類型及其他條件允許替換。寫新版本時保留舊target：

1. 建立唯一tmp並取得fd，依短寫規則寫完；設置需要的metadata。
2. fsync(tmpfd)成功，讓新file內容與必要metadata持久。
3. rename(tmp,target)成功；之後新lookup見新inode，已open舊inode者繼續讀舊版。
4. fsync(parent-dir-fd)成功後，才承諾新名稱在crash後持久；每步錯誤都需處理。

第3步的atomicity避免執行中出現「target暫時消失」；第4步關心持久名稱。尚未完成第4步時，本章不對所有filesystem宣稱固定回到old或new。兩步fsync也不是跨filesystem交易，rename跨filesystem可失敗EXDEV。

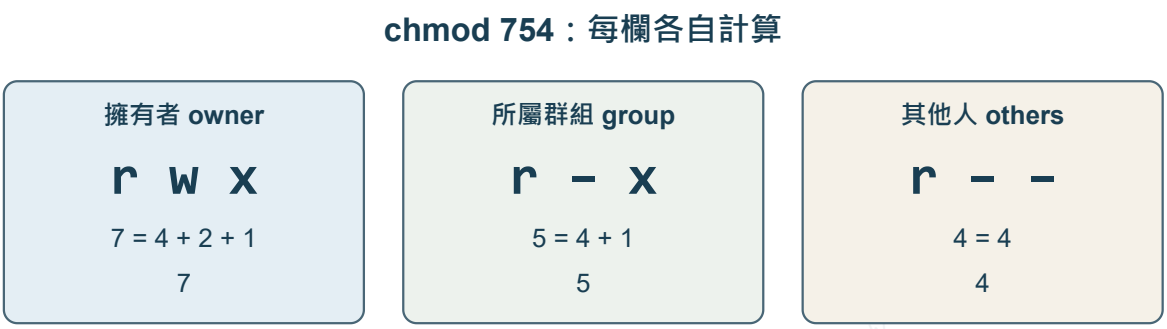
file rwx 與 directory rwx 的操作對象不同

忽略root/capability、ACL、sticky與其他保護，先看mode bits。Linux 以 filesystem UID（fsuid）、filesystem GID（fsgid）與 supplementary groups 作身分比對；本例假設 fsuid/fsgid 分別等於有效 UID/GID。依 owner 匹配、否則任一檔案系統群組匹配、否則 other 選「一個 class」，不把三class相加。bits r=4,w=2,x=1，要求位元req都在chosen中才准許。

權利	regular file	directory
r	讀取資料	列舉名稱
w	寫／改資料	新增、刪除、改名entry（通常另需x）
x	請求執行且格式等條件符合	search／穿越至已知名稱

D mode0711、F mode0644：非owner可穿越D找已知F並讀，不能列D全部名稱。D mode0733時other有wx，可新增／刪名，卻無r列名。刪F主要檢查父目錄wx，不是F的w；sticky directory另限制刪除者身分。

chmod 754：九個位元分成三組



`chmod 754 target` 將三組基本 mode bits 設為 owner `rwx`、group `r-x`、others `r--`。每組的 `r=4`、`w=2`、`x=1`：例如 group 的 `5=4+1`，所以有讀取與執行位元而沒有寫入位元。

權限檢查依適用的 owner、group 或 others 類別進行，不能把三欄任意相加。下例先假設 Linux 本機檔案、一般非特權使用者、沒有額外 ACL、LSM 或掛載限制；ACL、特殊權限和特權程序可能改變結果。

相同的 rwx，檔案與目錄代表什麼？

對象	r	w	x
普通檔案	允許讀取內容	允許寫入內容	可請求執行；格式、interpreter 等仍須符合
目錄	可列出名稱	可修改目錄項目，通常還須 x	可搜尋該目錄並解析其中名稱

假設 `/work` 是 754 目錄，沿途上層目錄均可搜尋。符合 group 類別者有 `r-x`，可列名稱、搜尋並走訪，但不能只靠這些權限在該目錄新增名稱；others 只有 `r--`，可讀取目錄項目名稱，卻缺少搜尋權，不能僅憑此解析 `/work/a`。owner 有 `rwx`；建立或刪除仍須符合其餘相關條件，例如 sticky bit 與掛載限制。

另一個普通檔案在 `chmod 754` 之前已由程序成功以可寫方式開啟。若後來修改 mode 移除該程序的寫入位元，Linux 本機一般檔案的已開 descriptor 不會因此自動關閉；新開啟是否成功則按當時權限判斷。此例不含額外 LSM 撤權、網路協定或裝置特殊行為；已開 fd 的 access mode 仍限制後續操作。

POSIX-style ACL：mask和選擇順序

以下為 Linux access ACL，非 Windows deny ACL；忽略 privilege/LSM。本例 filesystem UID/GID（fsuid/fsgid）分別等於 effective UID/GID，群組比對另包含 supplementary groups。owner=alice、filegroup=dev：user::rw-，user:bob:-w-，group::r--，group:qa:-w-，mask::rw-，other::r--。

1. fsuid 等於 owner，直接以user::判斷，不受mask。
2. 否則有matching named-user，要求其entry與mask都包含本次全部req；不通過便拒絕，不落到groups/other。
3. 否則匹配owning/named groups；至少一個matching group entry包含全部req，且mask包含req才准。已匹配卻不足就拒，不能落到other。
4. 沒有任何前述匹配，才用other；other不受mask。

Bob即使也是dev，只有w，不因dev而補r；把mask改r後，Bob完全無權，不回退other的r。Alice仍rw。ACL有mask時，顯示mode的group-class bits對應mask，不一定等於group::entry。

多group與複合請求：不能任意union後授權

令Carol屬dev及qa，沿前頁ACL，dev entry=r、qa entry=w、mask=rw。read(req4)可由dev通過；write(req2)可由qa通過；一次要求read+write(req6，如O_RDWR)卻沒有任何matching group entry同時含rw，因此拒絕。

主體／請求	命中路徑	判定
Alice／rw	owner rw	准；不套mask
Bob／r	named user w	拒；不回退groups
Bob／w	named user w & mask rw	准
Carol／r	dev r & mask rw	准
Carol／w	qa w & mask rw	准
Carol／rw	兩group各缺一項	拒
無matching身份／r	other r	准

將mask改r後，Carol的write也拒絕。此算法針對本次requested set，不是把所有顯示entry任意OR；named-user優先也不是一般化deny entry。

預設ACL、umask與既有handle

無default ACL時，create請求mode0666、umask0027，初始mode=0666 & ~0027 =0640；umask移除權利，不會把未要求的x加進去。

若父目錄有default ACL，子物件繼承為access ACL，再以create傳入mode限制對應owner/group-class/other權利；不能又無條件套一次umask。default ACL決定新物件初值，不會追溯修改舊file，也不是每次查詢的替代規則。

先open可寫fd，之後chmod0000：新open會被DAC擋，但一般Linux regular-file既有可寫fd通常仍可write。要撤回既有authority須定義不同機制／關閉或撤銷reference；修改畫面權限不等於回收所有已授權reference。ACL完整檢查規則見acl(5)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一檔案的四條追蹤線

追蹤線	必須記錄	常見錯誤
名稱	root/cwd、每個component、symlink suffix、mount map	先作字串link/..消去
物件	filesystem/inode、links、open references	unlink最後名稱就立刻free
存取	OFD offset/access mode、每次回傳長度	dup後假設offset獨立
授權	目錄search、物件ACL class、mask、授權時點	entry有w就無條件可寫

應用若要atomic publish再加持久線：tmp資料完成→file fsync→rename→directory fsync。名稱圖和持久儲存的提交時點不同，不能只用單一「成功」標籤表示。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從反例選擇正確操作

需求	不足的做法	能建立承諾的條件
指定offset讀且不影響共用游標	lseek後read	pread；仍處理短讀／錯誤
整筆append record不交錯	拆成多次O_APPEND write	record framing＋整筆鎖／單writer
刪除已知名稱	檢查file的w	父目錄wx、sticky與其他政策
限制解析到可信子樹	只檢查final不是symlink	可信root/reference及全路徑約束
跨client一致更新	只拿advisory lock	所有writer遵守＋cache/visibility協定
crash後新版本仍可找到	只rename成功	支援所述flush契約並完成file與dir持久化

本章的身分、名稱與handle契約，是第12章實作目錄／inode／cache時必須維持的介面；實作不能把物理block或cache狀態當成另一套使用者語意。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 11. Linux open(2); rename(2); acl(5); path_resolution(7); credentials(7).

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study